Department of Artificial Intelligence (AI) & Data Science

Name: Aaryan Nandkishor Raorane	**Roll no: S004**
Sap ID: 60018250070	**Topic: Data Structure**
Branch: Artificial Intelligence & Data Science	

Experiment No. 8

Aim: To implement Merge Sort and Quick Sort for sorting the product records of an online store by price in ascending order and by customer rating in descending order, and to compare the two techniques on comparisons, record moves and execution time.

Theory:

Definition:

Merge Sort is a sorting technique that follows the divide and conquer approach. It splits the array into two halves, sorts each half recursively and then merges the two sorted halves into one sorted array with the help of a temporary array. Its work does not depend on the order of the input.

Quick Sort is also a divide and conquer technique. It selects a pivot record, moves every record that belongs before the pivot to its left and the remaining records to its right, and then sorts both sides recursively. This program uses the last record of each range as the pivot.

A product record holds an id, a name, a price and a customer rating. The sorting routines never read price or rating directly. They call one function named isBefore, which uses a criterion variable to decide the field and the direction.

Operations:

a) **Compare:** It is used to decide whether one product must appear before another for the chosen criterion.

Pseudocode:

```
BEGIN isBefore(x, y)
    IF criterion == 1 THEN
        RETURN x.price < y.price
    ELSE
        RETURN x.rating > y.rating
    END IF
END isBefore
```

b) **Merge:** It is used to combine two sorted halves of the array into one sorted range using the temporary array.

Pseudocode:

```
BEGIN merge(a, low, mid, high)
    SET i = low
    SET j = mid + 1
    SET k = low
    WHILE i <= mid AND j <= high DO
        IF isBefore(a[j], a[i]) THEN
            temp[k] = a[j]
            j = j + 1
        ELSE
            temp[k] = a[i]
            i = i + 1
        END IF
        k = k + 1
    END WHILE
    WHILE i <= mid DO
        temp[k] = a[i]
        i = i + 1
        k = k + 1
    END WHILE
    WHILE j <= high DO
        temp[k] = a[j]
```

```
        j = j + 1
        k = k + 1
    END WHILE
    SET k = low
    WHILE k <= high DO
        a[k] = temp[k]
        k = k + 1
    END WHILE
END merge
```

c) **Merge Sort:** It is used to divide the range into halves, sort both halves and merge them.

Pseudocode:

```
BEGIN mergeSort(a, low, high)
    IF low < high THEN
        mid = (low + high) / 2
        mergeSort(a, low, mid)
        mergeSort(a, mid + 1, high)
        merge(a, low, mid, high)
    END IF
END mergeSort
```

d) **Partition:** It is used to place the pivot at its final position, with the records that come first on its left and the rest on its right.

Pseudocode:

```
BEGIN partition(a, low, high)
    pivot = a[high]
    i = low - 1
    SET j = low
    WHILE j < high DO
        IF isBefore(a[j], pivot) THEN
            i = i + 1
            SWAP(a[i], a[j])
        END IF
        j = j + 1
    END WHILE
    SWAP(a[i + 1], a[high])
```

```
        RETURN i + 1
    END partition
```

e) **Quick Sort:** It is used to partition the range and then sort the left and right parts recursively.

Pseudocode:

```
BEGIN quickSort(a, low, high)
    IF low < high THEN
        p = partition(a, low, high)
        quickSort(a, low, p - 1)
        quickSort(a, p + 1, high)
    END IF
END quickSort
```

The program also counts comparisons and record moves, and it times each sort over 20 repeated runs so that the time is large enough to measure. Sorted and reverse sorted inputs are built from the same random data by sorting it once and then reversing it.

Code:

Three arrays of 2000 records do not fit in one 64 KB segment, so Turbo C needs the large memory model for this program.

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 2000
#define REPEAT 20

struct product
{
   int id;
   char name[12];
   float price;
   float rating;
};

struct product original[MAX];
struct product work[MAX];
```

```
struct product temp[MAX];

int criterion;
long cmpCount, moveCount;

int isBefore(struct product *x, struct product *y)
{
   if (criterion == 1)
      return x->price < y->price;
   else
      return x->rating > y->rating;
}

void merge(struct product a[], int low, int mid, int high)
{
   int i = low, j = mid + 1, k = low;
   while (i <= mid && j <= high)
   {
      cmpCount++;
      if (isBefore(&a[j], &a[i]))
         temp[k++] = a[j++];
      else
         temp[k++] = a[i++];
      moveCount++;
   }
   while (i <= mid)
   {
      temp[k++] = a[i++];
      moveCount++;
   }
   while (j <= high)
   {
      temp[k++] = a[j++];
      moveCount++;
   }
   for (k = low; k <= high; k++)
   {
      a[k] = temp[k];
      moveCount++;
   }
}

void mergeSort(struct product a[], int low, int high)
{
   int mid;
   if (low < high)
   {
      mid = (low + high) / 2;
      mergeSort(a, low, mid);
      mergeSort(a, mid + 1, high);
      merge(a, low, mid, high);
```

```
    }
}

int partition(struct product a[], int low, int high)
{
    struct product pivot, t;
    int i, j;
    pivot = a[high];
    i = low - 1;
    for (j = low; j < high; j++)
    {
        cmpCount++;
        if (isBefore(&a[j], &pivot))
        {
            i++;
            t = a[i];
            a[i] = a[j];
            a[j] = t;
            moveCount += 3;
        }
    }
    t = a[i + 1];
    a[i + 1] = a[high];
    a[high] = t;
    moveCount += 3;
    return i + 1;
}

void quickSort(struct product a[], int low, int high)
{
    int p;
    if (low < high)
    {
        p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

void copyArray(struct product src[], struct product dst[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        dst[i] = src[i];
}

void reverseArray(struct product a[], int n)
{
    int i;
    struct product t;
    for (i = 0; i < n / 2; i++)
```

```
    {
        t = a[i];
        a[i] = a[n - 1 - i];
        a[n - 1 - i] = t;
    }
}

int isSorted(struct product a[], int n)
{
    int i;
    for (i = 0; i < n - 1; i++)
        if (isBefore(&a[i + 1], &a[i]))
            return 0;
    return 1;
}

void generateProducts(int n)
{
    int i;
    char category[5][12];
    strcpy(category[0], "Phone");
    strcpy(category[1], "Laptop");
    strcpy(category[2], "Shoes");
    strcpy(category[3], "Watch");
    strcpy(category[4], "Headset");
    srand((unsigned) time(NULL));
    for (i = 0; i < n; i++)
    {
        original[i].id = i + 1;
        strcpy(original[i].name, category[i % 5]);
        original[i].price = 100 + rand() % 99900;
        original[i].rating = 1.0 + (rand() % 41) / 10.0;
    }
}

void runTest(int n, char label[])
{
    int r, okMerge, okQuick;
    clock_t start, end;
    float mergeTime, quickTime;
    long mergeCmp, mergeMov, quickCmp, quickMov;

    start = clock();
    for (r = 0; r < REPEAT; r++)
    {
        copyArray(original, work, n);
        cmpCount = 0;
        moveCount = 0;
        mergeSort(work, 0, n - 1);
    }
    end = clock();
```

```
    mergeTime = ((float)(end - start) / CLOCKS_PER_SEC) * 1000 / REPEAT;
    mergeCmp = cmpCount;
    mergeMov = moveCount;
    okMerge = isSorted(work, n);

    start = clock();
    for (r = 0; r < REPEAT; r++)
    {
        copyArray(original, work, n);
        cmpCount = 0;
        moveCount = 0;
        quickSort(work, 0, n - 1);
    }
    end = clock();
    quickTime = ((float)(end - start) / CLOCKS_PER_SEC) * 1000 / REPEAT;
    quickCmp = cmpCount;
    quickMov = moveCount;
    okQuick = isSorted(work, n);

    printf("%-10s Merge %9ld %9ld %9.3f  %s\n", label, mergeCmp, mergeMov,
           mergeTime, okMerge ? "OK" : "FAIL");
    printf("%-10s Quick %9ld %9ld %9.3f  %s\n", "", quickCmp, quickMov,
           quickTime, okQuick ? "OK" : "FAIL");
}

void main()
{
    int n, i;
    clrscr();
    printf("Enter number of products (max %d): ", MAX);
    scanf("%d", &n);
    if (n < 2 || n > MAX)
        n = MAX;
    printf("1. Price ascending\n2. Rating descending\n");
    printf("Enter sorting criterion: ");
    scanf("%d", &criterion);
    generateProducts(n);

    printf("\n%-10s %-5s %9s %9s %9s\n", "Input", "Sort", "Compares",
           "Moves", "Time ms");
    runTest(n, "Random");

    mergeSort(original, 0, n - 1);
    runTest(n, "Sorted");

    reverseArray(original, n);
    runTest(n, "Reverse");

    copyArray(original, work, n);
    mergeSort(work, 0, n - 1);
    printf("\nFirst five records after sorting\n");
```

```
    for (i = 0; i < 5; i++)
        printf("%d %s %.0f %.1f\n", work[i].id, work[i].name,
            work[i].price, work[i].rating);
    getch();
}
```

Output:

Times depend on the machine and change a little from run to run. The comparison counts for sorted and reverse input stay almost the same.

```
Sorting 2000 Product Records by Price

Enter number of products (max 2000): 2000
1. Price ascending
2. Rating descending
Enter sorting criterion: 1

Input      Sort   Compares     Moves   Time ms
Random     Merge     19371     43904     0.380  OK
           Quick     24367     38112     0.288  OK
Sorted     Merge     11088     43904     0.199  OK
           Quick   1983274   5955699    19.954  OK
Reverse    Merge     10873     43904     0.215  OK
           Quick   1994618   2996388    13.287  OK

First five records after sorting
588 Shoes 177 3.8
1156 Phone 217 3.0
1035 Headset 240 1.8
692 Laptop 260 4.0
1218 Shoes 329 3.1
```

```
Sorting 2000 Product Records by Rating

Enter number of products (max 2000): 2000
1. Price ascending
2. Rating descending
Enter sorting criterion: 2

Input      Sort   Compares     Moves   Time ms
Random     Merge     19378     43904     0.323  OK
           Quick     61376     24195     0.382  OK
Sorted     Merge     11088     43904     0.188  OK
           Quick     88967    125658     0.603  OK
Reverse    Merge     12852     43904     0.204  OK
           Quick    128188    123198     0.737  OK

First five records after sorting
1996 Phone 37236 5.0
1872 Laptop 87753 5.0
1859 Watch 61644 5.0
1840 Headset 20283 5.0
1832 Laptop 57242 5.0
```

Applications:

- Used on product listing pages to order search results by price, rating or number of reviews.
- Used as the base of external sorting when data is too large to fit in memory, which is where Merge Sort is common.
- Used inside library sorting routines, where Quick Sort variants are common.
- Used for leaderboards, sales reports and top seller lists.

Conclusion:

Both programs sorted every input correctly for both criteria. On random data the two ran in nearly the same time. On sorted and reverse sorted price data, Quick Sort with the last record as pivot needed close to two million comparisons and ran roughly 100 and 60 times slower than Merge Sort in the sample run. Merge Sort stayed steady on every input, so it is the safer choice for an online store. Quick Sort needs a better pivot rule before it can be trusted on data that may already be ordered.